LECCIÓN ASINCRÓNICA · LUNES 26 DE OCTUBRE DE 2026

Funciones en C

Invocación, retorno, alcance y bibliotecas (math.h, ctype.h y stdlib.h)

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Capacidad	2.3 Emplear funciones en la resolución de problemas según las especificaciones técnicas.
Modalidad	Asincrónica · trabajo individual a su ritmo
Tiempo estimado	3 horas 45 minutos (vea la ruta de trabajo)
Herramientas	VS Code, gcc (MSYS2 UCRT64), Git Bash y su repositorio en GitHub
Entrega	Hoy lunes 26/10/2026, a más tardar a las __5__:_30pm___ (git push de la carpeta UA2/S21)
Persona facilitadora	Giovanni Antonio Coto Calderón

1. Ruta de trabajo de hoy

Hoy no hay clase presencial: usted avanza a su ritmo siguiendo esta ruta. Hágala **en orden**; cada paso prepara el siguiente.

Paso	Qué hacer	Tiempo
1	Leer la **explicación de la materia** (sección 2) y **probar cada ejemplo** en VS Code.	75 min
2	Ver el **video de referencia** (sección 3) con la guía de observación.	20 min
3	Responder las **preguntas de comprobación** (sección 4) en su README.	15 min
4	Resolver la **práctica autónoma** (sección 5): ejercicios 1, 2 y 3; el reto es opcional.	95 min
5	Completar el README, hacer los **commits**, el **git push** y la **autoevaluación** (secciones 6 y 7).	20 min

Al terminar la lección usted podrá

- Explicar qué es una función y para qué sirve dividir un programa en funciones.
- Escribir el **prototipo**, la **llamada** y la **definición** de una función, y diferenciar **parámetro** de **argumento**.
- Usar **return** y funciones **void**.
- Distinguir variables **locales**, **globales** y **static**.
- Usar funciones de biblioteca: **math.h**, **ctype.h** y **stdlib.h**.

2. Explicación de la materia

2.1 ¿Qué es una función?

Una **función** es un bloque de instrucciones **con nombre** que **recibe datos** (parámetros), **hace una tarea** y **puede devolver un resultado** (return). Se escribe **una vez** y se **llama** cuantas veces haga falta. Ya ha usado muchas: **printf**, **scanf**, **strlen** o **strcpy** son funciones que otra persona escribió; hoy aprende a escribir las suyas.

Piense en una función como una **receta**: tiene un nombre (Arroz con leche), unos ingredientes que recibe (parámetros), unos pasos (el cuerpo) y un plato que entrega (el valor de return). Quien la usa no necesita saber los pasos: solo la llama por su nombre y le da los ingredientes.

Ventaja	Qué significa
Dividir	Un problema grande se parte en tareas pequeñas, cada una con su función.
Reutilizar	Se escribe una vez y se usa muchas veces (por ejemplo, una validación).
Probar	Cada función se puede probar por separado con valores conocidos.
Leer	main queda corto y se lee como un índice de lo que hace el programa.

2.2 Prototipo, llamada y definición

Toda función propia aparece **tres veces** en el programa. El ejemplo calcula el IVA (13 %) de un precio:

iva.c

```c
#include <stdio.h>
#define TASA_IVA 0.13

double calcularIVA(double monto);   // prototipo

int main(void) {
    double precio = 15000;
    double iva = calcularIVA(precio);   // llamada con una variable
    printf("IVA de %.2f: %.2f\n", precio, iva);
    printf("IVA de 2500: %.2f\n", calcularIVA(2500));   // con un numero
    return 0;
}

double calcularIVA(double monto) {   // definicion
    return monto * TASA_IVA;
}
```

Salida real (gcc)

```
IVA de 15000.00: 1950.00
IVA de 2500: 325.00
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
1	`#include <stdio.h>`	Incluye la biblioteca de entrada y salida.	Copia las declaraciones de stdio.h antes de compilar.	Se puede usar printf.
2	`#define TASA_IVA 0.13`	Crea una constante.	Antes de compilar, cada TASA_IVA se cambia por 0.13; si cambia el impuesto, se toca un solo lugar.	—
4	`double calcularIVA(double monto);`	PROTOTIPO: anuncia la función.	Dice el tipo que devuelve (double), el nombre (calcularIVA) y lo que recibe: un parámetro llamado monto de tipo double. Termina en ; porque aquí no hay código. Va antes de main porque gcc lee de arriba hacia abajo.	gcc ya conoce la función cuando main la llama.
7	`double precio = 15000;`	Variable de main.	Guarda el precio que se va a enviar.	precio = 15000
8	`double iva = calcularIVA(precio);`	LLAMADA con un ARGUMENTO.	El valor de precio (15000) se COPIA en el parámetro monto y el programa salta a la definición (línea 14). Cuando la función hace return, la llamada se reemplaza por el valor devuelto.	iva = 1950
9	`printf("IVA de %.2f: %.2f\n", …);`	Muestra el resultado.	%.2f muestra un real con dos decimales.	IVA de 15000.00: 1950.00
10	`printf(…, calcularIVA(2500));`	LLAMADA con un número.	El argumento puede ser una variable, un número o una expresión. Aquí monto recibe 2500 y el resultado va directo a printf, sin guardarlo.	IVA de 2500: 325.00
14	`double calcularIVA(double monto) {`	DEFINICIÓN: el código de la función.	Mismo encabezado del prototipo, pero SIN ; y CON llaves. monto es una variable que existe solo dentro de esta función.	—
15	`return monto * TASA_IVA;`	Calcula y DEVUELVE.	return entrega el valor a quien llamó y termina la función.	15000 × 0.13 = 1950

¿Por qué primero dice double calcularIVA(double monto) y luego calcularIVA(precio)?

Son dos momentos distintos. En el **prototipo y la definición** se describe la función: por eso se escriben los **tipos** (double) y el nombre del **parámetro** (monto), que es como un espacio vacío que espera un dato.

En la **llamada** se usa la función: ya no se escriben tipos, solo se envía un **argumento**, el valor concreto (precio, 2500 o cualquier expresión). Ese valor se copia en monto.

Parámetro = la variable de la definición (monto). **Argumento** = el valor que se envía en la llamada (precio, 2500). Los nombres no tienen que coincidir: lo que importa es el **orden** y el **tipo**.

Pruébelo: copie el programa en VS Code como **iva.c**, compile con **gcc -Wall -Wextra iva.c -o iva** y ejecute con **./iva**.

2.3 Las partes de una función

Parte	Ejemplo	Para qué sirve
Tipo de retorno	`double calcularIVA(…)`	Qué tipo de dato devuelve (int, double, char…).
Nombre	`calcularIVA`	Verbo + qué hace. Sin tildes, sin espacios.
Parámetros	`(double monto)`	Datos que recibe. Son copias de los argumentos.
Cuerpo	`{ return monto * 0.13; }`	Las instrucciones de la tarea.
return	`return monto * 0.13;`	Devuelve el resultado y termina la función.
void	`void mostrarMenu(void)`	En el retorno: no devuelve nada. Entre paréntesis: no recibe nada.
Prototipo	`double calcularIVA(double monto);`	Anuncia la función antes de usarla (antes de main).
Argumento	`calcularIVA(15000)`	El valor que se envía en la llamada.

2.4 Funciones que no devuelven nada: void

Cuando una función solo **hace algo** (mostrar, dibujar) y no calcula un resultado, su tipo de retorno es **void** y no lleva return con valor. Además, una función puede **llamar a otra**: mostrarTitulo usa a mostrarLinea.

titulo.c

```c
#include <stdio.h>

void mostrarLinea(int largo);
void mostrarTitulo(void);

int main(void) {
    mostrarTitulo();
    return 0;
}

void mostrarLinea(int largo) {   // recibe el largo, no devuelve nada
    int i;
    for (i = 1; i <= largo; i++) {
        printf("-");
    }
    printf("\n");
}

void mostrarTitulo(void) {   // no recibe ni devuelve nada
    mostrarLinea(19);
    printf("COMPROBANTE DE PAGO\n");
    mostrarLinea(19);
}
```

Salida real (gcc)

```
------------------
COMPROBANTE DE PAGO
------------------
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
3–4	`void mostrarLinea(int largo); / void mostrarTitulo(void);`	Prototipos.	void al inicio = no devuelven nada. mostrarLinea recibe un int; mostrarTitulo no recibe nada (void entre paréntesis).	—
7	`mostrarTitulo();`	Llama a una función void.	Como no devuelve nada, la llamada va sola, sin guardar el resultado en una variable.	Se ejecuta mostrarTitulo.
11	`void mostrarLinea(int largo) {`	Definición de mostrarLinea.	largo recibe la copia del argumento (19).	largo = 19
13–14	`for (…) { printf("-"); }`	Dibuja guiones.	Repite largo veces; i es una variable LOCAL de esta función.	19 guiones
16	`printf("\n");`	Cambia de línea.	Para que lo siguiente empiece abajo.	—
20	`mostrarLinea(19);`	Una función llama a otra.	mostrarTitulo envía 19 a mostrarLinea; al terminar, vuelve a la línea siguiente.	----------
21	`printf("COMPROBANTE DE PAGO\n");`	Muestra el título.	Entre las dos líneas de guiones.	COMPROBANTE DE PAGO

Pruébelo: copie el programa en VS Code como **titulo.c**, compile con **gcc -Wall -Wextra titulo.c -o titulo** y ejecute con **./titulo**.

2.5 Una validación, muchos usos

La validación con **do … while** que usamos en la sesión 19 se escribe **una sola vez** dentro de una función, y se usa con **límites distintos** gracias a los parámetros.

leer.c

```c
#include <stdio.h>

int leerEntero(int minimo, int maximo);

int main(void) {
    int edad = leerEntero(15, 99);
    int opcion = leerEntero(0, 3);
    printf("Edad: %d  Opcion: %d\n", edad, opcion);
    return 0;
}

int leerEntero(int minimo, int maximo) {
    int valor;
    do {
        printf("Valor (%d a %d): ", minimo, maximo);
        scanf("%d", &valor);
    } while (valor < minimo || valor > maximo);
    return valor;
}
```

Salida real (se digita 120, 20, 5 y 2)

```
Valor (15 a 99): 120
Valor (15 a 99): 20
Valor (0 a 3): 5
Valor (0 a 3): 2
Edad: 20  Opcion: 2
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
3	`int leerEntero(int minimo, int maximo);`	Prototipo con dos parámetros.	Recibe dos int (el rango) y devuelve un int (el valor válido).	—
6	`int edad = leerEntero(15, 99);`	Primera llamada.	15 se copia en minimo y 99 en maximo (el ORDEN importa). Rechaza 120 y acepta 20.	edad = 20
7	`int opcion = leerEntero(0, 3);`	Segunda llamada, otro rango.	La MISMA función sirve para otro dato: ahora minimo = 0 y maximo = 3. Rechaza 5 y acepta 2.	opcion = 2
13	`int valor;`	Variable LOCAL.	Existe solo dentro de leerEntero; se crea en cada llamada.	—
14–17	`do { … } while (valor < minimo || valor > maximo);`	Valida.	Repite mientras el DATO esté fuera del rango recibido.	Rechaza 120 y 5.
18	`return valor;`	Devuelve el valor válido.	La llamada de main se reemplaza por este valor.	20, luego 2

Pruébelo: copie el programa en VS Code como **leer.c**, compile con **gcc -Wall -Wextra leer.c -o leer** y ejecute con **./leer**.

2.6 Alcance: variables locales y globales

El **alcance** es la parte del programa donde una variable existe. Una variable **local** se declara **dentro** de una función y solo existe ahí. Una variable **global** se declara **fuera de todas** las funciones y todas la ven. Dos funciones pueden tener variables locales con el **mismo nombre** sin mezclarse.

alcance.c

```c
#include <stdio.h>

int contador = 0;            // GLOBAL: fuera de todas las funciones

void registrar(void) {
    int temporal = 10;       // LOCAL de registrar
    contador++;
    printf("registrar: contador=%d temporal=%d\n", contador, temporal);
}

int main(void) {
    int temporal = 99;       // LOCAL de main (otra variable)
    registrar();
    registrar();
    printf("main: contador=%d temporal=%d\n", contador, temporal);
    return 0;
}
```

Salida real (gcc)

```
registrar: contador=1 temporal=10
registrar: contador=2 temporal=10
main: contador=2 temporal=99
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
3	`int contador = 0;`	Variable GLOBAL.	Está fuera de todas las funciones: main y registrar la pueden leer y cambiar.	contador = 0
6	`int temporal = 10;`	LOCAL de registrar.	Solo existe dentro de registrar.	temporal = 10
7	`contador++;`	Cambia la global.	El cambio se ve en todo el programa.	1, luego 2
12	`int temporal = 99;`	LOCAL de main.	Es OTRA variable, aunque se llame igual que la de registrar.	temporal de main = 99
15	`printf("main: …", contador, temporal);`	Muestra desde main.	Ve la global (2) y SU temporal (99), no la de registrar.	main: contador=2 temporal=99

Buena práctica

Evite las variables globales: cualquier función puede cambiarlas y los errores son difíciles de encontrar. Pase los datos como **parámetros** y devuelva los resultados con **return**.

2.7 Variables automáticas y static

Una variable local normal es **automática**: se **crea** en cada llamada y **desaparece** al salir de la función. Si se declara con **static**, se crea **una sola vez** y **conserva su valor** entre llamadas.

static.c

```c
#include <stdio.h>

void automatica(void) {
    int veces = 0;           // se crea en cada llamada
    veces++;
    printf("automatica: %d\n", veces);
}

void estatica(void) {
    static int veces = 0;    // se crea una sola vez
    veces++;
    printf("estatica:   %d\n", veces);
}

int main(void) {
    int i;
    for (i = 1; i <= 3; i++) {
        automatica();
        estatica();
    }
    return 0;
}
```

Salida real (gcc)

```
automatica: 1
estatica:   1
automatica: 1
estatica:   2
automatica: 1
estatica:   3
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
4	`int veces = 0;`	Local AUTOMÁTICA.	Se crea en 0 en CADA llamada.	Siempre muestra 1.
10	`static int veces = 0;`	Local STATIC.	Se inicializa en 0 una SOLA vez y conserva su valor al salir de la función.	Muestra 1, 2, 3.
17	`for (i = 1; i <= 3; i++) {`	Llama 3 veces a cada función.	Así se ve la diferencia entre las dos.	6 líneas

Ejemplo real: un contador de turnos («Atendiendo el turno 15») que vive dentro de la función que lo usa, sin necesidad de una variable global.

2.8 Funciones de biblioteca

C trae bibliotecas con funciones listas para usar. Solo hay que **incluirlas** con #include.

math.h · funciones numéricas

Función	Qué hace	Ejemplo	Resultado
`sqrt(x)`	Raíz cuadrada	`sqrt(144)`	12
`pow(b, e)`	Potencia (C no tiene el ^ de PSeInt)	`pow(2, 10)`	1024
`fabs(x)`	Valor absoluto de un real	`fabs(-7.5)`	7.5
`round(x)`	Redondea al entero más cercano	`round(2.5)`	3
`ceil(x)`	Redondea hacia arriba	`ceil(4.1)`	5
`floor(x)`	Redondea hacia abajo	`floor(4.9)`	4

Todas **reciben y devuelven double**, aunque el resultado sea entero: por eso se muestran con **%f** (%.0f quita los decimales). En MSYS2 (Windows) se compila igual que siempre; en Linux se agrega **-lm** al final del comando.

matematicas.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    printf("sqrt(144)  = %.2f\n", sqrt(144));
    printf("pow(2, 10) = %.0f\n", pow(2, 10));
    printf("fabs(-7.5) = %.1f\n", fabs(-7.5));
    printf("round(2.5) = %.0f\n", round(2.5));
    printf("ceil(4.1)  = %.0f\n", ceil(4.1));
    printf("floor(4.9) = %.0f\n", floor(4.9));
    return 0;
}
```

Salida real (gcc)

```
sqrt(144)  = 12.00
pow(2, 10) = 1024
fabs(-7.5) = 7.5
round(2.5) = 3
ceil(4.1)  = 5
floor(4.9) = 4
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
2	`#include <math.h>`	Incluye la biblioteca matemática.	Sin esta línea gcc no conoce sqrt, pow, etc.	—
6	`printf("… %.0f\n", pow(2, 10));`	Potencia.	pow(base, exponente) devuelve un double; %.0f lo muestra sin decimales.	1024
8	`round(2.5)`	Redondeo.	Las mitades se alejan de cero.	3
9–10	`ceil(4.1) / floor(4.9)`	Hacia arriba / hacia abajo.	ceil sirve cuando no se puede comprar «media caja»: 10 / 4.0 = 2.5 → ceil = 3 cajas.	5 y 4

ctype.h · funciones de caracteres

isalpha, **isdigit** e **isspace** preguntan qué tipo de carácter es (devuelven distinto de 0 si es verdad y 0 si no). **toupper** y **tolower** lo convierten a mayúscula o minúscula.

caracteres.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char texto[] = "Ced 1-0234";
    int i, letras = 0, digitos = 0;
    for (i = 0; texto[i] != '\0'; i++) {
        if (isalpha(texto[i])) {
            letras++;
        }
        if (isdigit(texto[i])) {
            digitos++;
        }
        putchar(toupper(texto[i]));
    }
    printf("\nLetras: %d  Digitos: %d\n", letras, digitos);
    return 0;
}
```

Salida real (gcc)

```
CED 1-0234
Letras: 3  Digitos: 5
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
5	`char texto[] = "Ced 1-0234";`	Cadena con valor inicial.	Sin número entre corchetes, C calcula el tamaño (10 caracteres + '\0').	—
7	`for (i = 0; texto[i] != '\0'; i++) {`	Recorre la cadena.	Avanza carácter por carácter hasta el '\0' (fin de la cadena).	10 vueltas
8	`if (isalpha(texto[i]))`	¿Es letra?	isalpha devuelve verdadero para A–Z y a–z.	C, e, d → 3
11	`if (isdigit(texto[i]))`	¿Es dígito?	isdigit devuelve verdadero para 0–9.	1, 0, 2, 3, 4 → 5
14	`putchar(toupper(texto[i]));`	Escribe en mayúscula.	toupper convierte la letra; lo que no es letra queda igual. putchar escribe un carácter.	CED 1-0234

stdlib.h · rand, srand, abs y atoi

rand() da un número pseudoaleatorio; **rand() % 6 + 1** simula un dado (LanzarDado, sesión 2). **srand** fija la semilla. **abs** da el valor absoluto de un entero y **atoi** convierte un texto en número.

azar.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i;
    srand(2026);                          // semilla fija
    for (i = 1; i <= 5; i++) {
        printf("%d ", rand() % 6 + 1);    // dado de 1 a 6
    }
    printf("\nabs(-8) = %d\n", abs(-8));
    printf("atoi(\"2026\") + 1 = %d\n", atoi("2026") + 1);
    return 0;
}
```

Salida real (Linux; en Windows los números del dado pueden ser otros)

```
2 1 2 4 5
abs(-8) = 8
atoi("2026") + 1 = 2027
```

Explicación línea a línea

Línea	Instrucción	¿Qué hace?	¿Cómo y por qué?	Efecto
2	`#include <stdlib.h>`	Incluye la biblioteca estándar.	Trae rand, srand, abs y atoi.	—
6	`srand(2026);`	Fija la semilla.	Con la misma semilla se repite la misma secuencia en el mismo sistema: útil para probar.	—
8	`printf("%d ", rand() % 6 + 1);`	Lanza el dado.	rand() da un entero grande; % 6 deja el residuo 0..5 y + 1 lo convierte en 1..6.	2 1 2 4 5
10	`abs(-8)`	Valor absoluto.	Para enteros (fabs es para reales).	8
11	`atoi("2026") + 1`	Texto a número.	Convierte la cadena "2026" en el entero 2026, que ya se puede sumar.	2027

2.9 Errores frecuentes (y cómo leerlos)

Error	Qué dice gcc	Corrección
Llamar una función sin prototipo antes de main	implicit declaration of function 'calcularIVA' y conflicting types for 'calcularIVA' (en gcc 14 de MSYS2 es un error)	Escribir el prototipo antes de main.
Un camino sin return en una función que devuelve valor	warning: control reaches end of non-void function	Agregar un return en todos los caminos (por ejemplo, return 0; al final).
Usar una variable local fuera de su función	error: 'total' undeclared (first use in this function)	Devolver el valor con return y guardarlo en main.
Olvidar el #include de la biblioteca	implicit declaration of function 'sqrt'	Agregar #include <math.h> (o la que corresponda).

Error	Qué dice gcc	Corrección
Poner ; al final de la definición	error: expected identifier or '(' before '{' token	La definición va SIN ; (solo el prototipo lo lleva).

Ejemplo del segundo error: si el total es menor que 50000, la función termina **sin devolver nada** y el resultado es impredecible.

sin_return.c (con error)

```c
#include <stdio.h>

double calcularDescuento(double total) {
    if (total >= 50000) {
        return total * 0.10;
    }
}

int main(void) {
    printf("%.2f\n", calcularDescuento(60000));
    return 0;
}
```

Mensaje de gcc

```
sin_return.c:7:1: warning: control reaches end of non-void function [-Wreturn-type]
```

3. Video de referencia

Vea el video **después** de leer la explicación: así lo usa para repasar y aclarar. Puede pausarlo y probar el código en VS Code.

Video principal (15 min)

Manejo de funciones y procedimientos en C · canal Leopoldo Morales · 15:16 · en español.

https://www.youtube.com/watch?v=zjJ7eF_8oPE

Video complementario (opcional)

Aprende C en 45 minutos (curso desde cero) · canal Programador X · vea la parte de **Funciones** como repaso general.

https://www.youtube.com/watch?v=1-qjMS6C960

Guía de observación (anote las respuestas; las usará en la sección 4):

1. ¿Qué diferencia hace el video entre una **función** y un **procedimiento**? ¿Cómo se escribe un procedimiento en C?
2. ¿Dónde coloca el autor el prototipo y dónde la definición?
3. Anote un ejemplo del video en el que una función **recibe datos** y otro en el que **devuelve** un resultado.
4. ¿Qué tiene en común el video con iva.c o leer.c de esta lección?

Si un enlace no abre, busque el título en YouTube.

4. Preguntas de comprobación

Responda con sus palabras en la sección **«Comprobación»** de su **UA2/S21/README.md**. No copie la explicación: si puede decirlo con sus palabras, lo entendió.

1. ¿Qué diferencia hay entre el **parámetro** y el **argumento**? Use calcularIVA(double monto) y calcularIVA(precio) para explicarlo.
2. ¿Qué hace **return** y qué pasa con las líneas que están después de él?
3. ¿Cuándo una función es **void**? Dé un ejemplo propio.
4. En alcance.c, ¿por qué main muestra temporal=99 si registrar puso temporal = 10?
5. ¿Qué diferencia hay entre una variable local **automática** y una **static**?
6. ¿Cómo se calcula 2 elevado a 10 en C y qué biblioteca se necesita?
7. Del video: escriba **una idea** que le haya aclarado algo de la lección.

5. Práctica autónoma

Instrucciones generales

- Trabajo **individual**. Puede consultar esta lección y el video. Si pide ayuda, que sea para entender, no para copiar (valor: honradez).
- Guarde todo en la carpeta **UA2/S21/** de su portafolio, con los nombres indicados (sin tildes, sin ñ y sin espacios).
- Al inicio de cada archivo escriba en comentarios el **enunciado**, **qué se logra**, **cómo se logra** y el **resultado esperado**. Cada función lleva un comentario con **qué recibe, qué hace y qué devuelve**. Mensajes de printf **sin tildes ni ñ**.
- Cada programa debe compilar con **gcc -Wall -Wextra** sin errores ni advertencias.

Antes de empezar

1. En Git Bash: **cd /c/INA/<su-repositorio>** y **git pull**.
2. Cree la carpeta: **mkdir -p UA2/S21** y ábrala en VS Code.
3. Compile así: **gcc -Wall -Wextra conversiones.c -o conversiones** y ejecute con **./conversiones**.

Recordatorio

Necesito…	En C
Anunciar una función	`double calcularIVA(double monto);`
Definirla	`double calcularIVA(double monto) { return monto * 0.13; }`
Una función sin resultado	`void mostrarLinea(int largo) { … }`
Usar el resultado	`iva = calcularIVA(precio);`
Bibliotecas	`#include <math.h>  <ctype.h>  <stdlib.h>`

Ejercicio 1 · Biblioteca de conversiones → conversiones.c (30 min)

Qué se logra: reunir en funciones cuatro algoritmos de la UA1 y comprobar que funcionan con un programa de prueba.

Elementos que se practican: prototipos, parámetros y return; funciones que devuelven double y funciones que devuelven 1 o 0.

Función	Algoritmo de la UA1	Prueba
`double celsiusAFahrenheit(double c)`	ConversionTemperatura (S02): F = C × 9 / 5 + 32	25 → 77.0
`double colonesADolares(double c)`	ConversorMoneda (S11): 510 colones por dólar	5100 → 10.00
`int esBisiesto(int anio)`	AnioBisiesto (S08)	2024 → 1, 1900 → 0

Función	Algoritmo de la UA1	Prueba
`int esNotaValida(double nota)`	ValidarNota (S12): de 0 a 100	105 → 0, 85 → 1

main es un **programa de prueba**: llama a cada función con el valor de la tabla y muestra el resultado junto al esperado.

Ejecución esperada en la terminal

```
25 C = 77.0 F (esperado 77.0)
5100 colones = 10.00 dolares (esperado 10.00)
2024 bisiesto: 1 (esperado 1)
1900 bisiesto: 0 (esperado 0)
Nota 105 valida: 0 (esperado 0)
Nota 85 valida: 1 (esperado 1)
```

Pista: escriba 9.0 / 5.0 (con punto) para evitar la división entera. Una condición en C ya vale 1 o 0: return nota >= 0 && nota <= 100; no necesita if.

Ejercicio 2 · LanzarDado (sesión 2) → dado.c (30 min)

Qué se logra: simular 600 lanzamientos de un dado y mostrar cuántas veces salió cada cara con una barra de asteriscos.

Elementos que se practican: rand y srand de stdlib.h, una función que devuelve un valor (**int lanzarDado(void)**), otra void (**void mostrarBarra(int cantidad)**, un * por cada 10 lanzamientos) y un arreglo de contadores (sesión 20).

Ejecución de referencia (Linux, semilla 2026; en Windows las cantidades pueden variar)

```
Cara 1: 120 ************
Cara 2:  77 *******
Cara 3: 108 **********
Cara 4:  97 *********
Cara 5: 103 **********
Cara 6:  95 *********
```

Pista: use int frecuencia[7] = {0}; y la cara como índice: frecuencia[cara]++. Así no necesita un if por cada cara.

Ejercicio 3 · Analizar una frase → texto.c (25 min)

Qué se logra: leer una frase, mostrarla en mayúsculas y contar vocales, consonantes y dígitos.

Elementos que se practican: getchar y putchar (sesión 20), isalpha, isdigit, toupper y tolower de ctype.h, y una función propia **int esVocal(int c)** que devuelve 1 o 0.

Ejecución esperada en la terminal

```
Frase: Sede INA 2026
SEDE INA 2026
Vocales: 4  Consonantes: 3  Digitos: 4
```

Pista: dentro de esVocal convierta el carácter con tolower y compare solo con 'a', 'e', 'i', 'o' y 'u'.

Reto opcional · Redondeo a 5 colones → redondeo.c (10 min)

Escriba **double redondearCinco(double monto)** que redondee un monto al múltiplo de 5 más cercano con **round**: 1232 → 1230; 1233 → 1235; 1237.5 → 1240.

6. Entrega y evidencia

Archivo	Contenido
`UA2/S21/conversiones.c`	Ejercicio 1.
`UA2/S21/dado.c`	Ejercicio 2.
`UA2/S21/texto.c`	Ejercicio 3.
`UA2/S21/redondeo.c`	Reto (opcional).
`UA2/S21/README.md`	Respuestas de la sección 4 («Comprobación»), una captura de la ejecución de cada ejercicio y la autoevaluación.
`Historial de commits`	Un commit por ejercicio (S21: agrega conversiones.c, …), otro para el README y git push al final.

Recuerde

- Marque la **S21** con ☑ en el registro de evidencias y agregue la fila del día en la **bitácora** («Lección asincrónica, 26/10/2026»).
- La fecha y hora de sus commits muestran que trabajó hoy: **no deje todo para el final**.
- Si tiene dudas, escríbale a la persona facilitadora por el medio habitual del curso, indicando el ejercicio y el mensaje de gcc.

7. Autoevaluación

Marque con X según corresponda y cópiela en su README. Sea honesto: sirve para que usted y la persona facilitadora sepan qué reforzar.

Indicador	Sí	Aún no
Escribo el prototipo de cada función antes de main.		
Diferencio el parámetro (en la definición) del argumento (en la llamada).		
Elijo el tipo de retorno correcto (double, int o void).		
Cada camino de una función con resultado tiene return.		
Uso parámetros en lugar de variables globales.		
Distingo una variable local, global y static.		
Uso funciones de math.h, ctype.h y stdlib.h.		
Mis programas compilan con -Wall -Wextra sin advertencias.		